In [1]:

```
import pandas as pd
```

Read me first!

As part of your interview process, you may be asked a question that involves writing python in a jupyter notebook and using pandas to work with the data involved in this question. (Rest assured that we will take your background and experiences into account when scheduling your final round interview -- if you've never written python and never done data science you probably won't be asked this type of question!)

In case you don't have much (or any) pandas experience, this primer is provided for you to look over. It goes over some pandas basics, along with some useful pandas functionality. If you are asked this type of question, this primer will also be available to you during the interview, in case you want to reference it.

You don't need to memorize this content (or anything close to that)!

We won't be trying to test your ability to memorize how Pandas works, or your ability to do things in Python! If you are asked this type of question, you would have this primer available during the interview, and your interviewer would be happy to help you write any code you don't know how to write.

What is pandas?

Pandas is a python framework that makes data manipulation much easier. Pandas is at the core of most studies we do at Jane Street, so having some familiarity with it is helpful!

The core type in pandas is a DataFrame (often referred to as a df). A df is essentially a table -- it has some columns and some rows, and you can do all kinds of data manipulation and visualization on it.

Creating a df

For your interview, you'll be reading dataframes from csv files. For example:

In [2]:
```
df = pd.read_csv('data.csv')
df
```

Out[2]:

	date	product	category	price	quantity	customer_id
0	2023-01-15	Laptop	Electronics	999.99	1	C001
1	2023-01-16	T-shirt	Clothing	19.99	3	C002
2	2023-01-16	Sneakers	Footwear	79.95	1	C003
3	2023-01-17	Blender	Appliances	49.99	2	C004
4	2023-01-18	Book	Books	14.50	5	C002
5	2023-01-19	Smartphone	Electronics	599.00	1	C005
6	2023-01-20	Jeans	Clothing	39.99	2	C001
7	2023-01-21	Toaster	Appliances	29.95	1	C003
8	2023-01-22	Headphones	Electronics	89.99	1	C004
9	2023-01-23	Dress	Clothing	59.95	1	C005

Series

Every column of a df is a pd.Series, but you can more or less think of it as a list

In [3]:
```
df["price"]
```

Out[3]:

```
0    999.99
1     19.99
2     79.95
3     49.99
4     14.50
5    599.00
6     39.99
7     29.95
8     89.99
9     59.95
Name: price, dtype: float64
```

In [4]:

```
# you can usually treat a series like a list
[ p+1 for p in df["price"] ]
```

Out[4]:

```
[1000.99, 20.99, 80.95, 50.99, 15.5, 600.0, 40.99, 30.95, 90.99, 60.9
5]
```

Visualization

Filtering

Filtering columns

In [5]:

```
df[["price", "category"]]
```

Out[5]:

	price	category
0	999.99	Electronics
1	19.99	Clothing
2	79.95	Footwear
3	49.99	Appliances
4	14.50	Books
5	599.00	Electronics
6	39.99	Clothing

7	29.95	Appliances
8	89.99	Electronics
9	59.95	Clothing

Filtering rows

In [6]:

```
# boolean operations on a series produce a boolean series
(df["price"] > 20) & (df["category"] != 'Electronics')
```

Out[6]:

```
0    False
1    False
2     True
3     True
4    False
5    False
6     True
7     True
8    False
9     True
dtype: bool
```

In [7]:

```
# you can filter a df on a boolean series
df[(df["price"] > 20) & (df["category"] != 'Electronics')]
```

Out[7]:

	date	product	category	price	quantity	customer_id
2	2023-01-16	Sneakers	Footwear	79.95	1	C003
3	2023-01-17	Blender	Appliances	49.99	2	C004
6	2023-01-20	Jeans	Clothing	39.99	2	C001
7	2023-01-21	Toaster	Appliances	29.95	1	C003
9	2023-01-23	Dress	Clothing	59.95	1	C005

In [8]:

```
# you can also use df.query
df.query('(price > 20) and (category != "Electronics")')
```

Out[8]:

	date	product	category	price	quantity	customer_id
2	2023-01-16	Sneakers	Footwear	79.95	1	C003
3	2023-01-17	Blender	Appliances	49.99	2	C004
6	2023-01-20	Jeans	Clothing	39.99	2	C001
7	2023-01-21	Toaster	Appliances	29.95	1	C003
9	2023-01-23	Dress	Clothing	59.95	1	C005

Plotting

There are lots of ways to make plots in python, and you can use whatever you like for your interview! Here we provide some sample code using plotly, which is a python plotting library some folks at Jane Street like to use.

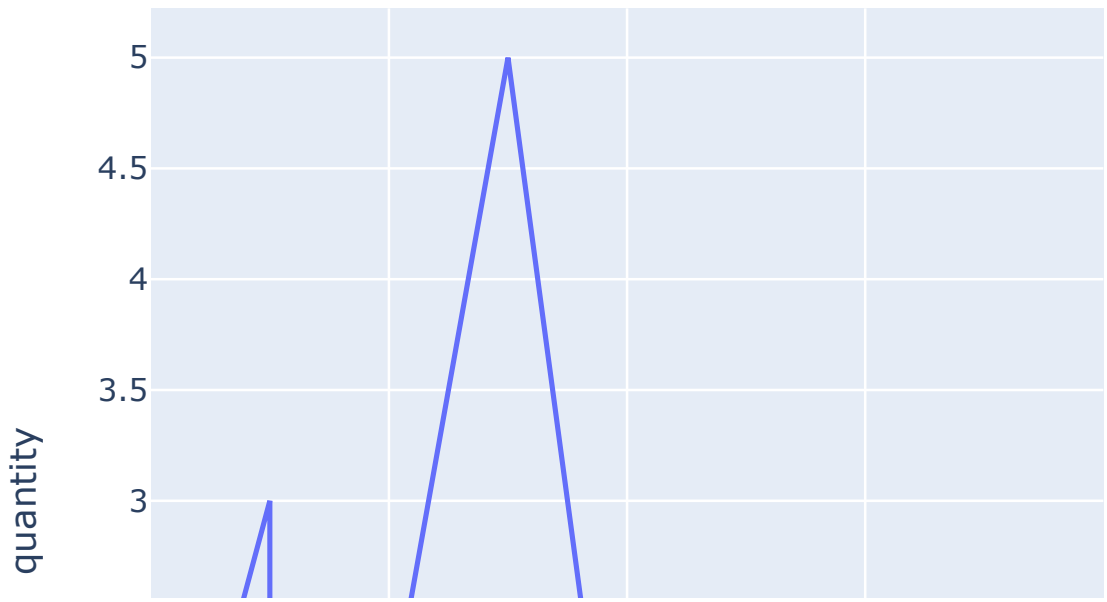
In [9]:

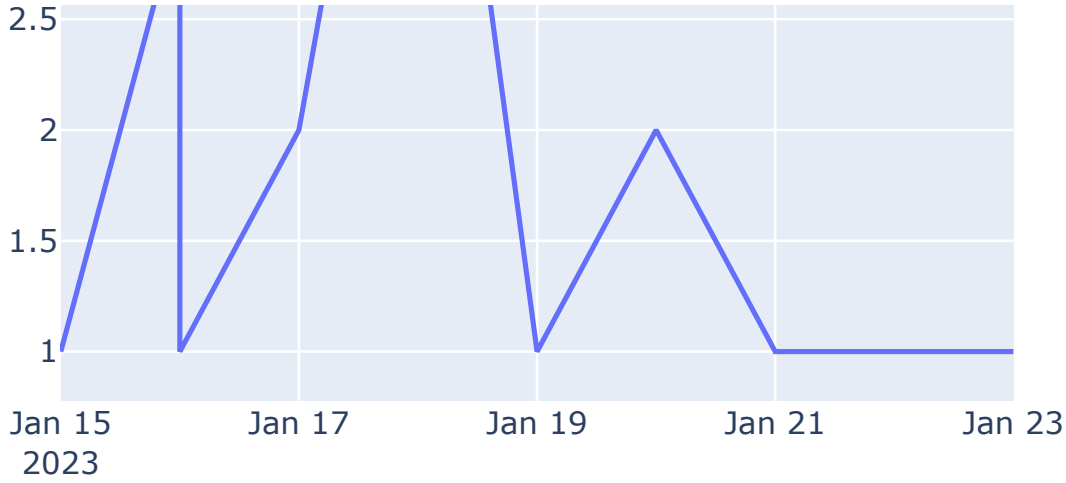
```
import plotly.express as px
```

Line Plot

In [10]:

```
px.line(df, x="date", y="quantity")
```

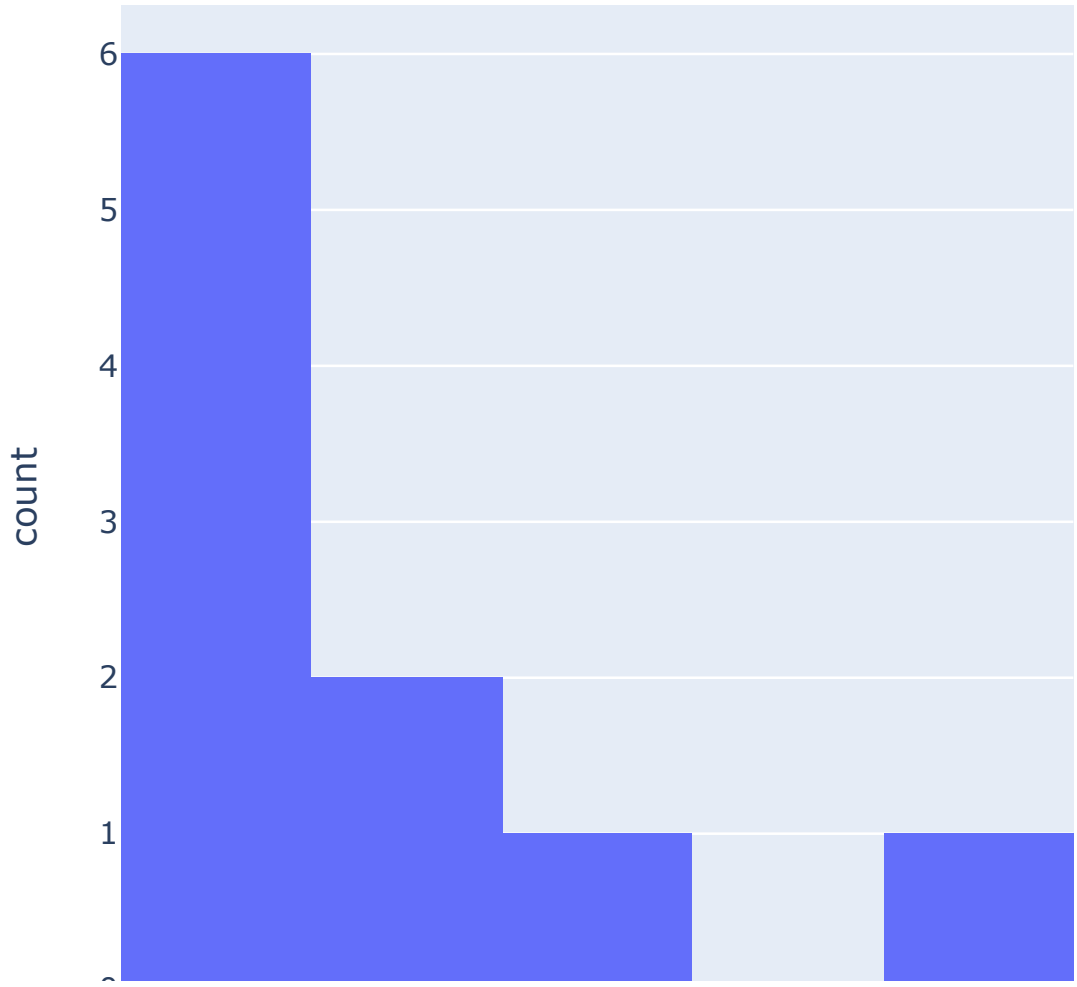


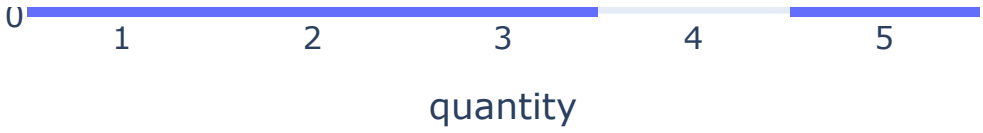


Histogram

In [11]:

```
px.histogram(df,x='quantity')
```

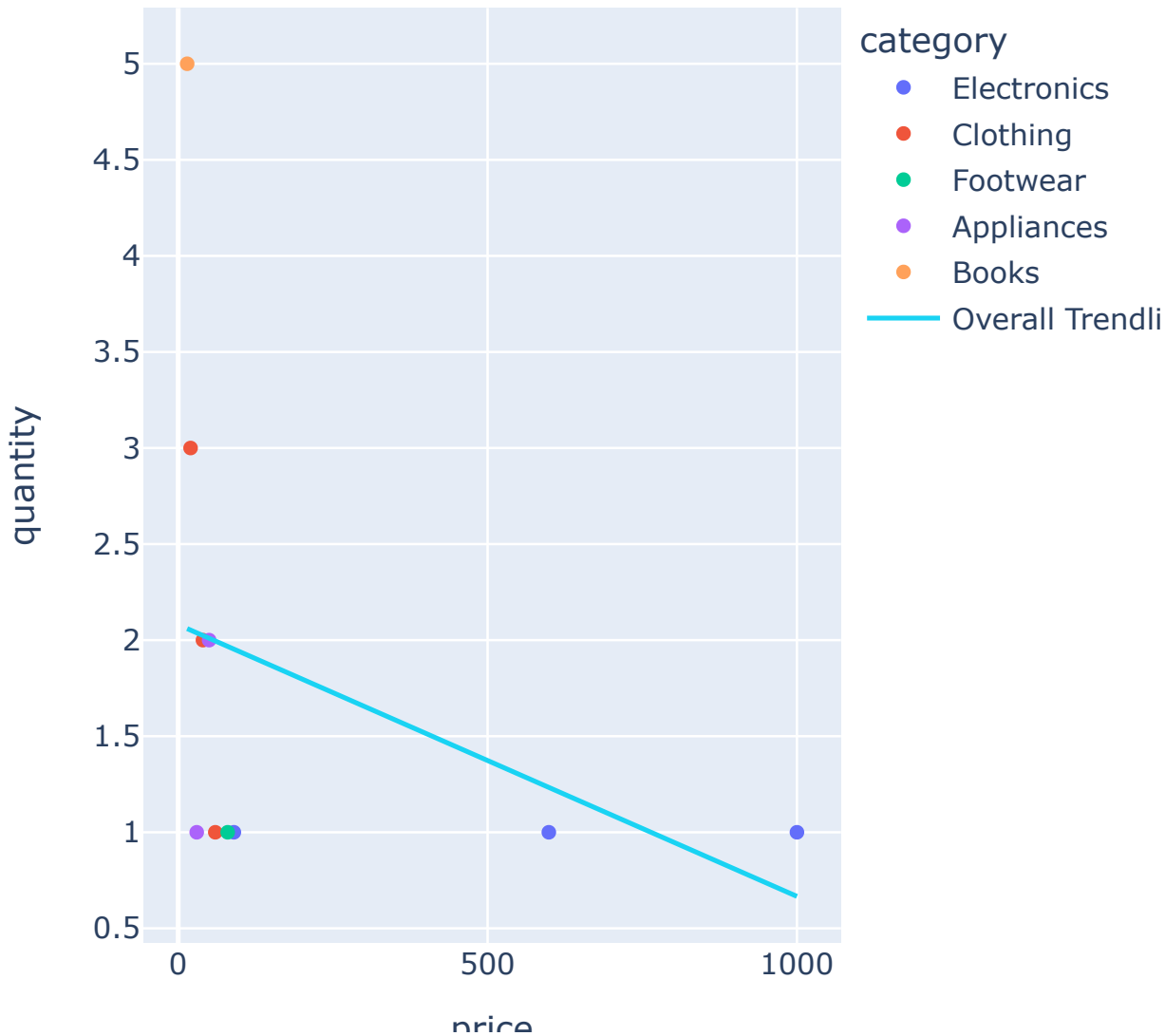




Scatterplot

In [12]:

```
px.scatter(
    df,
    x="price",
    y="quantity",
    color="category",
    trendline="ols",
    trendline_scope="overall",
)
```



price

Describe

df.describe() outputs useful summary stats of your df. It's helpful to get a broad sense of what your data looks like.

In [13]:
```
df.describe()
```

Out[13]:

	price	quantity
count	10.000000	10.000000
mean	198.330000	1.800000
std	331.515438	1.316561
min	14.500000	1.000000
25%	32.460000	1.000000
50%	54.970000	1.000000
75%	87.480000	2.000000
max	999.990000	5.000000

Sorting

In [14]:
```
df.sort_values("quantity")
```

Out[14]:

	date	product	category	price	quantity	customer_id
0	2023-01-15	Laptop	Electronics	999.99	1	C001
2	2023-01-16	Sneakers	Footwear	79.95	1	C003
5	2023-01-19	Smartphone	Electronics	599.00	1	C005
7	2023-01-21	Toaster	Appliances	29.95	1	C003

8	2023-01-22	Headphones	Electronics	89.99	1	C004
9	2023-01-23	Dress	Clothing	59.95	1	C005
3	2023-01-17	Blender	Appliances	49.99	2	C004
6	2023-01-20	Jeans	Clothing	39.99	2	C001
1	2023-01-16	T-shirt	Clothing	19.99	3	C002
4	2023-01-18	Book	Books	14.50	5	C002

In [15]:

```
# you can pass a list to sort by multiple columns
df.sort_values(["quantity", "price"])
```

Out[15]:

	date	product	category	price	quantity	customer_id
7	2023-01-21	Toaster	Appliances	29.95	1	C003
9	2023-01-23	Dress	Clothing	59.95	1	C005
2	2023-01-16	Sneakers	Footwear	79.95	1	C003
8	2023-01-22	Headphones	Electronics	89.99	1	C004
5	2023-01-19	Smartphone	Electronics	599.00	1	C005
0	2023-01-15	Laptop	Electronics	999.99	1	C001
6	2023-01-20	Jeans	Clothing	39.99	2	C001
3	2023-01-17	Blender	Appliances	49.99	2	C004
1	2023-01-16	T-shirt	Clothing	19.99	3	C002
4	2023-01-18	Book	Books	14.50	5	C002

Data manipulation

Adding columns

In [16]:

```
df["total_dollars_spent"] = df["price"] * df["quantity"]
```

```
df
```

Out[16]:

	date	product	category	price	quantity	customer_id	total_doll
0	2023-01-15	Laptop	Electronics	999.99	1	C001	
1	2023-01-16	T-shirt	Clothing	19.99	3	C002	
2	2023-01-16	Sneakers	Footwear	79.95	1	C003	
3	2023-01-17	Blender	Appliances	49.99	2	C004	
4	2023-01-18	Book	Books	14.50	5	C002	
5	2023-01-19	Smartphone	Electronics	599.00	1	C005	
6	2023-01-20	Jeans	Clothing	39.99	2	C001	
7	2023-01-21	Toaster	Appliances	29.95	1	C003	
8	2023-01-22	Headphones	Electronics	89.99	1	C004	
9	2023-01-23	Dress	Clothing	59.95	1	C005	

Mapping over a column

In [17]:

```
df["year"] = df["date"].map(lambda x: x.split("-")[0])
df
```

Out[17]:

	date	product	category	price	quantity	customer_id	total_doll
0	2023-01-15	Laptop	Electronics	999.99	1	C001	
1	2023-	T-shirt	Clothing	19.99	3	C002	

	01-16					
2	2023-01-16	Sneakers	Footwear	79.95	1	C003
3	2023-01-17	Blender	Appliances	49.99	2	C004
4	2023-01-18	Book	Books	14.50	5	C002
5	2023-01-19	Smartphone	Electronics	599.00	1	C005
6	2023-01-20	Jeans	Clothing	39.99	2	C001
7	2023-01-21	Toaster	Appliances	29.95	1	C003
8	2023-01-22	Headphones	Electronics	89.99	1	C004
9	2023-01-23	Dress	Clothing	59.95	1	C005

groupby

groupby aggregates together rows of your df that have the same values for a given set of columns

In [18]:

```
# get the average price and total dollars spent by category
df.groupby("category").aggregate({'price': 'mean', 'total_dollars_spe
```

Out[18]:

category	price	total_dollars_spent
Appliances	39.970000	129.93
Books	14.500000	72.50
Clothing	39.976667	199.90
Electronics	562.993333	1688.98

Footwear	79.950000	79.95

In [19]:

```
# we'll use groupby -> count to see how many rows exist in the df for
# all the other columns in the df are equal for a given row since it'
df.groupby("category").count().reset_index()
```

Out[19]:

	category	date	product	price	quantity	customer_id	total_dollars_sp
0	Appliances	2	2	2	2	2	
1	Books	1	1	1	1	1	
2	Clothing	3	3	3	3	3	
3	Electronics	3	3	3	3	3	
4	Footwear	1	1	1	1	1	

In [20]:

```
# get the total dollars spent by date
df.groupby("date")["total_dollars_spent"].sum().reset_index()
```

Out[20]:

	date	total_dollars_spent
0	2023-01-15	999.99
1	2023-01-16	139.92
2	2023-01-17	99.98
3	2023-01-18	72.50
4	2023-01-19	599.00
5	2023-01-20	79.98
6	2023-01-21	29.95
7	2023-01-22	89.99
8	2023-01-23	59.95

Merging

Merging two dfs is analogous to a join in sql -- take the cartesian product of the two dfs and keep only the rows for which the given columns' values all match.

This is useful when you're trying to stitch together different data sources, or pull a piece of information from some other dataset into your main working df.

In [21]:
```
customer_df = pd.read_csv("customer_data.csv")
customer_df
```

Out[21]:

	customer_id	name	age	city	membership_type
0	C001	John Smith	38	New York	Gold
1	C002	Emma Johnson	22	Los Angeles	Silver
2	C003	Michael Brown	41	Chicago	Bronze
3	C004	Sophia Lee	29	San Francisco	Gold
4	C005	Robert Chen	45	Houston	Silver
5	C006	Olivia Davis	21	Boston	Bronze
6	C007	William Taylor	56	Seattle	Gold
7	C008	Ava Wilson	31	Miami	Silver
8	C009	James Anderson	29	Denver	Bronze
9	C010	Isabella Martinez	37	Phoenix	Gold

In [22]:
```
pd.merge(df, customer_df, on="customer_id")
```

Out[22]:

	date	product	category	price	quantity	customer_id	total_doll
0	2023-01-15	Laptop	Electronics	999.99	1	C001	
1	2023-01-20	Jeans	Clothing	39.99	2	C001	

2	2023-01-16	T-shirt	Clothing	19.99	3	C002
3	2023-01-18	Book	Books	14.50	5	C002
4	2023-01-16	Sneakers	Footwear	79.95	1	C003
5	2023-01-21	Toaster	Appliances	29.95	1	C003
6	2023-01-17	Blender	Appliances	49.99	2	C004
7	2023-01-22	Headphones	Electronics	89.99	1	C004
8	2023-01-19	Smartphone	Electronics	599.00	1	C005
9	2023-01-23	Dress	Clothing	59.95	1	C005